



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE LA RECHERCHE ET DE L'INNOVATION

Rapport de Projet

de

Deuxième année du premier semestre

Présenté et soutenu publiquement le 10/12/2024

Par

ELLOUMI Molka

MENJLI Fakhri

Valise Autonome

Membre du jury

Monsieur

JEBRI Ayoub

Monsieur

SIMOND Nicolas

Année universitaire : 2024-2025

Table des matières

Introduction générale	1
Chapitre 1: Gestion de Projet	2
Introduction	2
1 Cahier des Charges	2
2 Étude de l’Existant	3
3 Limites des Solutions Existantes	4
4 Proposition de Notre Solution	4
5 Analyse Comparative : Solutions Existantes vs Notre Solution	5
6 Diagramme de Gantt	6
Conclusion	6
Chapitre 2: Environnement de travail	7
Introduction	7
1 Environnement matériel	7
2.1.1 Le microcontrôleur ESP32	7
2.1.2 Moteurs CC à engrenages FIT0564	8
2.1.3 Capteur ultrasonique SH04	9
2.1.4 Contrôleur de moteur (PCB personnalisé)	9
2.1.5 Alimentation	10
Batterie Li-ion	10
Batterie 12V	10
2 Environnement logiciel	11
2.2.1 Proteus 8 ISIS	11

2.2.2	Arduino IDE	12
2.2.3	Flutter	12
2.2.4	KiCad	13
2.2.5	Visual Studio Code (VS Code)	13
	Conclusion	13
Chapitre 3:	ARCHITECTURE DU SYSTÈME	15
	Introduction	15
1	Schéma d'ensemble	15
2	Flux de Communication	16
3	Fonctionnement Global de la Valise	17
	Conclusion	17
Chapitre 4:	Réalisation logicielle	18
1	Programmation embarquée de l'ESP32	18
4.1.1	Lecture des capteurs	18
4.1.2	Estimation de la distance via le RSSI	19
4.1.3	Logique de navigation intelligente	19
4.1.4	Logique de détection d'obstacles avec tolérance	20
4.1.5	Tests	21
2	Application mobile Flutter	21
4.2.1	Présentation générale et fonctionnalités	21
4.2.2	Architecture et logique de communication	22
4.2.3	Interface utilisateur	24
Chapitre 5:	Réalisation matérielle	27
1	Le contrôleur de moteur	27
5.1.1	Fonctionnement du contrôleur	27
5.1.2	Dimensionnement du Contrôleur Moteur	29
	Composants Sélectionnés	29
	Simulation du Contrôleur	29
	Essai du circuit sur plaque d'essai	30
5.1.3	Routage du PCB sur KiCad	31
5.1.4	Impression du PCB et tests	33
2	Integration des Capteurs et alimentation	35

5.2.1	Capteur ultrasonique SH04	35
	Calcul de la distance	35
3	Châssis et structure mécanique	36
5.3.1	Conception et matériau du châssis	36
5.3.2	Dimensionnement et implantation des composants	37
	Bibliographie	40

Table des figures

1.1	Diagramme de Gantt global de la deuxième phase du projet	6
2.1	Le microcontrôleur ESP32	8
2.2	Moteurs CC à engrenages FIT0564	9
2.3	Capteur ultrasonique SH04	9
2.4	PCB personnalisé	10
2.5	batterie 12v	11
2.6	Proteus 8 ISIS	11
2.7	Arduino IDE	12
2.8	Flutter	12
2.9	KiCad	13
2.10	VS Code	13
3.1	Schéma d'architecture globale du système	15
4.1	Aperçu des résultats de tests	21
4.2	Diagramme de cas d'utilisation de l'application mobile Flutter	22
4.3	Architecture MVVM utilisée dans notre application Flutter	23
4.4	Structure des fichiers de l'application Flutter	24
4.5	Exemple de contrôle de saisie dans l'écran de connexion	25
4.6	Interface principale de suivi (hors connexion)	26
5.1	Arrêt du moteur lorsque PWM1 et PWM2 sont nuls	28
5.2	Fonctionnement avec PWM1 actif	28
5.3	Fonctionnement avec PWM2 actif	28
5.4	Fonctionnement avec PWM1 et PWM2 actifs	28

5.5	Tableau récapitulatif du fonctionnement du notre controlleur	28
5.6	Simulation : Contrôle bidirectionnel du moteur	30
5.7	Essai du circuit de contrôle moteur	31
5.8	Routage du PCB sur KiCad	32
5.9	largeur des pistes	33
5.10	les tests	34
5.11	Capteur ultrasonique SH04	36

Introduction générale

Le projet de valise autonome vise à répondre aux besoins des voyageurs dans des environnements variés comme les aéroports, les gares ou les trottoirs. Cette innovation cherche à réduire les efforts physiques liés au transport des bagages tout en améliorant la mobilité et le confort des utilisateurs. Avec l'évolution des technologies de navigation et de suivi, ce projet explore une solution autonome qui combine électronique avancée et intelligence embarquée pour offrir une expérience pratique et innovante.

Gestion de Projet

Introduction

Ce chapitre du projet présente la gestion de notre travail. Nous y détaillons tout d'abord le cahier des charges, suivi d'une étude des solutions existantes sur le marché, qui nous a permis de nous inspirer tout en identifiant leurs limites. Nous proposons ensuite notre solution et terminons par l'élaboration du diagramme de Gantt pour la première phase du projet, puis celui relatif à la seconde phase.

1 Cahier des Charges

Dans cette section, nous allons parler du cahier des charges. À partir du sujet proposé, nous avons pu déterminer ses fonctionnalités obligatoires et élaborer un cahier des charges servant de référence pour notre travail. Nous avons également élaboré le tableau ci-dessous. Ce tableau comporte les fonctionnalités, leur expression, les critères, et les moyens de contrôle.

- Expression : il s'agit de l'explication de la fonctionnalité.
- Critères : ils désignent les conditions ou exigences spécifiques que la fonctionnalité doit remplir pour être considérée comme réussie. Un critère représente ce qui est attendu comme résultat pour vérifier la fonctionnalité.
- Moyens de contrôle : ce sont les outils ou méthodes permettant de s'assurer que les critères sont respectés.

Fonctionnalité (FP)	Expression	Critères	Moyen de Contrôle
FP1 : Suivi Autonome	Permettre à la valise de suivre automatiquement son propriétaire.	La valise doit détecter la position du propriétaire en temps réel.	Capteurs ultrasoniques, un module wifi intégré dans la carte ESP32, et les moteurs.
FP2 : Navigation	Naviguer de manière autonome dans différents environnements.	Capacité à détecter et à éviter des obstacles immobiles et mobiles.	Capteurs ultrasoniques, un module wifi intégré dans la carte ESP32 et les moteurs.
FP3 : Mode Manuel	Permettre le contrôle manuel de la valise.	Le mode manuel doit être activé facilement et rapidement.	Interface utilisateur (boutons, application).
FP4 : Notifications	Envoyer des alertes au propriétaire en cas de perte de connexion.	Notification sur smartphone en cas de perte de connexion.	WIFI
FP5 : Autonomie de la Batterie	Fonctionner sans recharge fréquente.	Autonomie d'au moins 8 heures en utilisation normale.	Batterie lithium-ion

TABLE 1.1 – Spécifications des fonctionnalités principales du projet

2 Étude de l'Existant

Après des recherches, nous avons pu identifier trois solutions similaires à notre projet. Bien que nous n'ayons pas trouvé de références techniques, nous avons trouvé des descriptions des technologies utilisées, ce qui nous a aidés à mieux comprendre. Voici un tableau qui donne un aperçu de ces solutions en présentant précisément les technologies utilisées, leurs avantages et leurs inconvénients.

Solution	Technologie Utilisée	Avantages	Inconvénients
Valise Ovis	Suivi par caméra et IA	- Suivi précis et intelligent - Bonne navigation dans les environnements ouverts	- Très coûteuse (prix élevé). - Autonomie limitée (batterie).

Valise Travel-mate	Suivi par Bluetooth (RSSI)	- Faible coût relatif - Facile à mettre en œuvre	- Navigation imprécise en cas d'obstacles. - Portée limitée de la connexion Bluetooth.
Valise Cowarobot	Capteurs ultrasoniques et LiDAR	- Excellente détection d'obstacles - Bonne autonomie de batterie	- Lourde et coûteuse. - Complexité dans la maintenance.

3 Limites des Solutions Existantes

À travers cette analyse, nous constatons que :

- Le **coût élevé** des solutions comme **Ovis** ou **Cowarobot** limite leur accessibilité.
- Les technologies basées uniquement sur le **Bluetooth** montrent des limites en termes de **portée** et de **précision**.
- Les systèmes complexes intégrant des **caméras IA** ou **LiDAR** augmentent la consommation énergétique, ce qui réduit l'autonomie de la batterie.

Nous avons pu, à travers ces observations, élaborer notre solution en tenant compte de nos contraintes, tout en envisageant d'apporter des améliorations et des optimisations. Notre solution doit :

- Utiliser des technologies abordables .
- Maintenir un **coût réduit** tout en respectant les contraintes techniques et budgétaires.
- Assurer une **autonomie énergétique satisfaisante** en intégrant une batterie adaptée.

4 Proposition de Notre Solution

Pour mieux présenter ce que nous souhaitons réaliser, nous avons élaboré un tableau qui résume comment chaque fonctionnalité discutée dans le cahier des charges sera satisfaite technologiquement à travers la solution technologique proposée, ainsi qu'une justification brève de ce choix, tout en respectant une contrainte budgétaire stricte.

Composant	Technologie Choisie	Justification
Suivi du propriétaire	Wi-Fi (RSSI)	Solution abordable et plus robuste que Bluetooth pour une communication fiable.
Détection d'obstacles	Capteurs ultrasoniques HC-SR04	Solution abordable et fiable pour la navigation et la détection d'obstacles.
Navigation et contrôle	Microcontrôleur ESP32	Intègre Wi-Fi pour la communication et Bluetooth comme option secondaire, à faible coût.
Propulsion	Moteurs CC avec un contrôleur	Économique, fiable et facile à contrôler.
Alimentation	Batterie lithium-ion rechargeable	Offre une bonne autonomie pour une utilisation prolongée avec un poids réduit.

TABLE 1.3 – Composants et technologies choisis pour la solution

5 Analyse Comparative : Solutions Existantes vs Notre Solution

Finalement, nous avons réalisé ce tableau récapitulatif qui met en valeur ce que nous avons amélioré par rapport aux autres solutions.

Critère	Valise Ovis	Valise Travelmate	Notre Solution
Technologie de Suivi	IA + Caméra	Bluetooth	Wifi (RSSI)
Technologie de Navigation	Caméras/LiDAR	Capteurs basiques	Capteurs ultrasoniques HC-SR04
Coût	Très élevé	Modéré	Abordable
Autonomie	Moyenne	Bonne	Optimisée avec batterie Li-Ion
Précision de Suivi	Élevée	Faible	Moyenne (améliorable par optimisation)

Facilité de mise en œuvre	Complexe	Simple	Simple
----------------------------------	----------	--------	--------

6 Diagramme de Gantt

Le diagramme de Gantt est un outil de planification visuelle permettant de structurer les tâches d'un projet dans le temps. Pour la seconde phase de notre projet, nous avons élaboré un diagramme global afin de définir les différentes étapes prévues et organiser leur déroulement.

Cependant, en raison de divers problèmes techniques et de contraintes rencontrés en cours de réalisation, nous n'avons pu mener à bien que les **trois dernières tâches** planifiées.



FIGURE 1.1 – Diagramme de Gantt global de la deuxième phase du projet

Conclusion

Ce chapitre a permis de présenter les grandes lignes de notre projet, d'introduire notre cahier des charges ainsi que notre réflexion autour de la solution retenue. Nous avons également expliqué notre planification, tout en précisant les limites rencontrées lors de la mise en œuvre. Le chapitre suivant détaillera l'environnement matériel et logiciel utilisé pour concrétiser notre solution.

Environnement de travail

Introduction

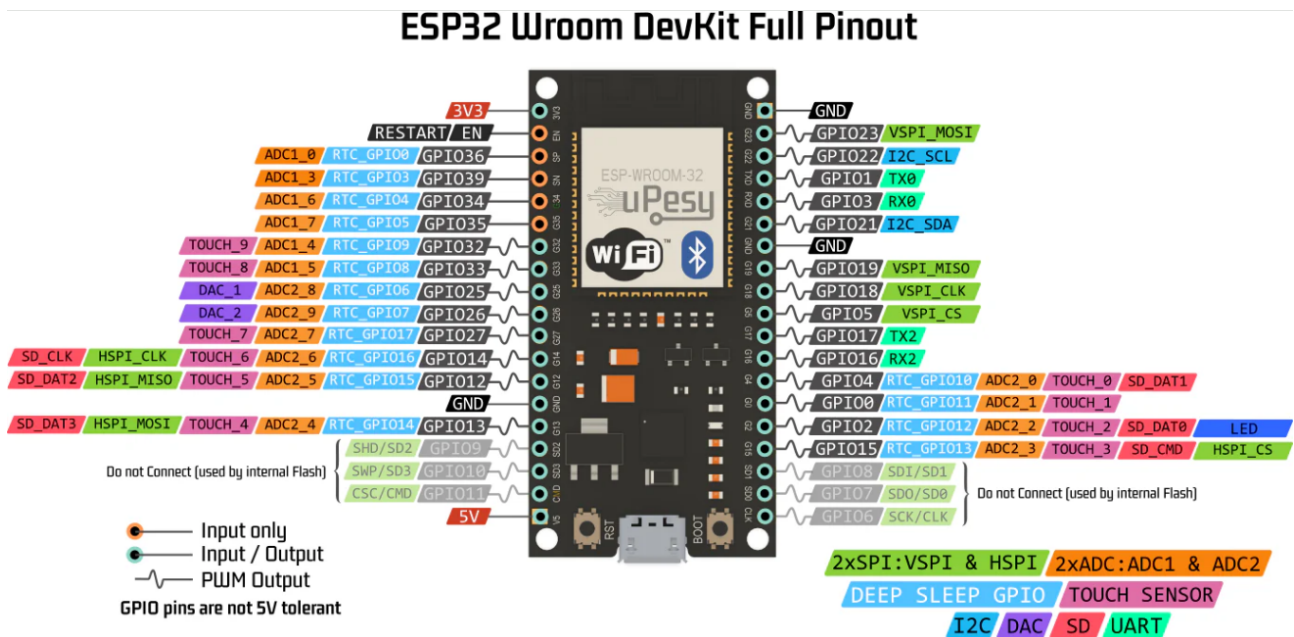
Ce chapitre décrit les différents environnements de travail, tant matériels que logiciels, qui ont été nécessaires à la réalisation de ce projet. Ces environnements ont été soigneusement sélectionnés pour répondre aux exigences techniques et fonctionnelles de la solution envisagée.

1 Environnement matériel

Pour la conception et la réalisation de la valise autonome, plusieurs éléments matériels ont été utilisés :

2.1.1 Le microcontrôleur ESP32

Le microcontrôleur ESP32 a été choisi pour ses performances élevées en termes de traitement et de connectivité. Il dispose d'un processeur double cœur, d'une connectivité Wi-Fi et Bluetooth intégrée, ce qui le rend idéal pour gérer les communications sans fil et les interactions avec les capteurs et les moteurs. Sa faible consommation énergétique et son large écosystème de support logiciel en font un choix fiable pour les applications embarquées.



2.1.2 Moteurs CC à engrenages FIT0564

Ces moteurs à courant continu avec réducteur intégré offrent une combinaison idéale de puissance et de précision. Le réducteur permet de fournir un couple élevé, nécessaire pour déplacer la valise, tout en maintenant une vitesse contrôlée. Leur robustesse et leur durabilité en font un choix adapté pour une utilisation prolongée dans des conditions variées.

TABLE 2.1 – Étude mathématique pour le choix du moteur

Étape	Prototype (1 kg)	Cas réel (23 kg)
1. Force exercée par la charge	$f = m \cdot g = 9.81 \text{ N}$	$f = m \cdot g = 225.63 \text{ N}$
2. Couple	$C = f \cdot r = 0.4905 \text{ Nm}$	$C = f \cdot r = 11.2815 \text{ Nm}$
3. Vitesse angulaire	$N = \frac{1.460}{2\pi \cdot 0.05} \approx 134.5 \text{ tr/min}$ $= 2\pi \frac{N}{60} \approx 14.1 \text{ rad/s}$	$N = \frac{1.460}{2\pi \cdot 0.05} \approx 134.5 \text{ tr/min}$
4. Puissance	$P = 6.91 \text{ W}$	$P = 159 \text{ W}$
5. Courant	$I = \frac{P}{V} = \frac{6.91}{12} \approx 0.7 \text{ A}$	$I = \frac{P}{V} = \frac{159}{48} \approx 3 \text{ A}$
6. Durée de fonctionnement	$t = \frac{\text{capacité}}{\text{courant}} = \frac{9}{1.15} \approx 8 \text{ heures}$	$t = \frac{\text{capacité}}{\text{courant}} = \frac{18}{3} \approx 6 \text{ heures}$

Les calculs ci-dessus ont été réalisés afin de choisir le moteur adapté en prenant en compte la force, le couple, la vitesse angulaire, la puissance nécessaire et l'autonomie énergétique.



FIGURE 2.2 – Moteurs CC à engrenages FIT0564

2.1.3 Capteur ultrasonique SH04

Ce capteur est utilisé pour mesurer les distances grâce à l'émission et la réception d'ondes ultrasoniques. Avec une portée précise allant jusqu'à plusieurs mètres, il permet à la valise de détecter et d'éviter les obstacles tout en suivant son propriétaire. Sa simplicité d'intégration et sa fiabilité en font un composant essentiel pour la navigation autonome.

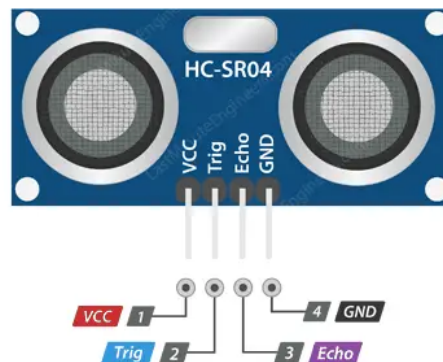


FIGURE 2.3 – Capteur ultrasonique SH04

2.1.4 Contrôleur de moteur (PCB personnalisé)

Le contrôleur de moteur est un circuit imprimé conçu spécifiquement pour ce projet. Il intègre un hacheur à quatre quadrants, permettant de contrôler à la fois la vitesse et le sens de rotation des moteurs. Ce circuit a été conçu pour offrir une grande précision et une efficacité énergétique optimale. Sa conception sur mesure garantit une compatibilité parfaite avec les moteurs et les autres composants du système.



FIGURE 2.4 – PCB personnalisé

2.1.5 Alimentation

L'alimentation de la valise autonome repose sur deux systèmes distincts : une batterie Li-ion pour alimenter la carte ESP32 et les capteurs, et une batterie 12V pour les moteurs.

Batterie Li-ion

Cette batterie, d'une capacité suffisante, alimente la carte ESP32 et les capteurs. Elle est choisie pour sa haute densité énergétique, permettant une autonomie prolongée. Un régulateur de tension permet de convertir la sortie de la batterie Li-ion (3.7V) en 3.3V pour l'ESP32.

Batterie 12V

Elle est utilisée pour alimenter les moteurs et le contrôleur de moteur. Cette batterie est dimensionnée pour fournir une autonomie suffisante lors des déplacements de la valise.



FIGURE 2.5 – batterie 12v

2 Environnement logiciel

Le développement et la simulation du projet ont nécessité l'utilisation de deux outils logiciels :

2.2.1 Proteus 8 ISIS

Cet outil a été employé pour la conception et la simulation des circuits électroniques. Grâce à sa capacité de simuler des composants en temps réel, il a permis de tester et d'optimiser le schéma du contrôleur de moteur avant sa fabrication. La simulation a inclus des tests de tension, de courant et de réponse dynamique des circuits.

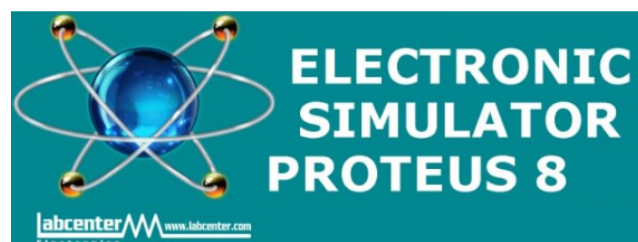


FIGURE 2.6 – Proteus 8 ISIS

2.2.2 Arduino IDE

L'Arduino Integrated Development Environment (IDE) a été utilisé pour écrire, compiler et téléverser le code sur l'ESP32. Cet outil propose une interface intuitive qui simplifie le développement grâce à une vaste bibliothèque de fonctions prêtes à l'emploi, comme la gestion des capteurs, des moteurs et de la connectivité. L'IDE offre également un moniteur série intégré, utile pour le débogage en temps réel des fonctionnalités du système.



FIGURE 2.7 – Arduino IDE

2.2.3 Flutter

Flutter est un framework open source développé par Google permettant de créer des applications mobiles multiplateformes. Dans le cadre de ce projet, Flutter a été utilisé pour développer une application mobile Android servant d'interface de communication avec la valise autonome.

Grâce à son langage Dart et son environnement de développement intégré, Flutter a permis de concevoir des interfaces utilisateur modernes et réactives.



FIGURE 2.8 – Flutter

2.2.4 KiCad

KiCad est un logiciel de conception assistée par ordinateur (CAO) libre utilisé pour concevoir des circuits imprimés (PCB). Dans notre projet, KiCad a été utilisé pour la réalisation du schéma électrique et du routage du PCB du contrôleur moteur bidirectionnel.



FIGURE 2.9 – KiCad

2.2.5 Visual Studio Code (VS Code)

Visual Studio Code est un éditeur de code multiplateforme largement utilisé pour le développement embarqué. Dans notre projet, il a été utilisé pour écrire et organiser le code destiné à l'ESP32, notamment à l'aide de l'extension PlatformIO, qui offre une meilleure gestion des projets et des dépendances que l'IDE Arduino.

VS Code a également permis d'éditer les scripts en Dart pour Flutter, de tester les requêtes HTTP, et de configurer les paramètres réseau du projet. Son interface moderne et ses nombreuses extensions ont grandement facilité le processus de développement et de débogage.



FIGURE 2.10 – VS Code

Conclusion

Nous avons bien décrit dans ce chapitre l'environnement de travail mis en place pour ce projet, qui repose sur une combinaison d'équipements matériels robustes et d'outils logiciels perfor-

ments. Dans le prochain chapitre, nous aborderons l'architecture de notre valise.

ARCHITECTURE DU SYSTÈME

Introduction

Ce chapitre présente l'architecture globale de la solution technique mise en place pour la réalisation de la valise autonome. Nous y décrivons l'organisation matérielle et logicielle, les flux de communication entre les différents modules, ainsi que le fonctionnement global du système. Une attention particulière est portée à l'interaction avec l'utilisateur, qui reçoit en temps réel des informations cruciales via l'application mobile Flutter.

1 Schéma d'ensemble

Le diagramme suivant illustre l'interaction entre les principaux composants matériels et leur rôle dans le fonctionnement de la valise autonome.

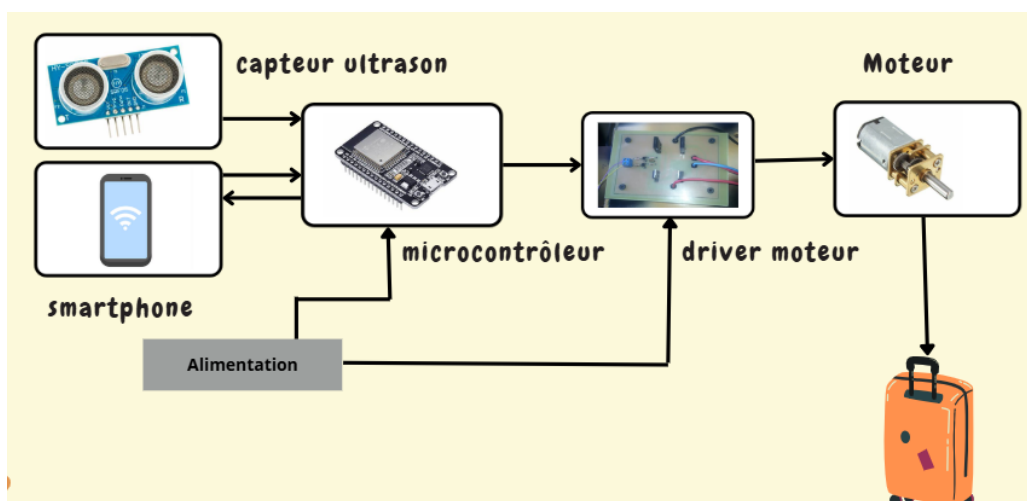


FIGURE 3.1 – Schéma d'architecture globale du système

Les éléments principaux du système sont :

- **Microcontrôleur (ESP32)** : Coordonne le système en traitant les données des capteurs et en contrôlant les moteurs.
- **Capteurs ultrasoniques** : Détectent les obstacles à proximité et transmettent les distances au microcontrôleur.
- **Contrôleur de moteur** : Régule la puissance et la direction des moteurs selon les instructions du microcontrôleur.
- **Moteurs à engrenages CC** : Assurent le mouvement et la direction de la valise.
- **Connexion WiFi** : Permet la communication entre la valise et le smartphone de l'utilisateur.
- **Batterie lithium-ion** : Fournit l'énergie nécessaire au fonctionnement des composants.

2 Flux de Communication

Les flux de données entre les différents composants garantissent le fonctionnement global du système. Voici la séquence des échanges :

1. **Capteurs vers Microcontrôleur** : Les capteurs ultrasoniques mesurent les distances aux obstacles et envoient les informations au microcontrôleur (ESP32).
2. **Microcontrôleur vers Contrôleur de Moteur** : Le microcontrôleur analyse les données reçues et envoie des instructions précises au contrôleur pour ajuster la vitesse et la direction des moteurs.
3. **Contrôleur de Moteur vers Moteurs** : Le contrôleur de moteur régule la puissance fournie aux moteurs pour exécuter les commandes du microcontrôleur (mouvement, arrêt ou changement de direction).
4. **Microcontrôleur vers Application Smartphone** : Le microcontrôleur utilise la connexion WiFi pour transmettre des informations essentielles à l'application Flutter installée sur le smartphone de l'utilisateur. Cette communication inclut :
 - le statut de la connexion entre la valise et l'utilisateur (valise connectée ou perdue),
 - la distance estimée entre la valise et le smartphone (via RSSI),
 - la détection éventuelle d'un obstacle (via le capteur ultrasonique).

L'application affiche ces informations de manière intuitive, permettant à l'utilisateur de réagir rapidement en cas de problème et de suivre la position de la valise en toute simplicité.

3 Fonctionnement Global de la Valise

L'algorithme global qui régit le fonctionnement de la valise peut être décrit en quatre étapes principales :

1. **Détection et Connexion WiFi** : La valise établit une connexion WiFi avec le smartphone de l'utilisateur. Le microcontrôleur récupère en continu les informations sur la densité du signal WiFi pour estimer la position relative du propriétaire.
2. **Suivi de la Direction** : Si le signal WiFi diminue, la valise détecte qu'elle s'éloigne et ajuste sa trajectoire en conséquence. Les moteurs à engrenages CC sont commandés pour corriger la direction.
3. **Gestion des Obstacles** : Les capteurs ultrasoniques mesurent en temps réel les distances avec les obstacles environnants. En cas de détection, le microcontrôleur ajuste les moteurs pour éviter la collision.
4. **Mise à Jour de la Direction** : La valise combine les informations des capteurs et du signal WiFi pour corriger continuellement sa trajectoire, assurant ainsi un suivi fluide et sécurisé.

Conclusion

Ce chapitre a présenté l'architecture globale du système, en mettant en lumière les interactions entre les composants matériels, logiciels, et l'utilisateur. L'intégration de l'application Flutter assure une communication bidirectionnelle efficace, permettant à la valise d'agir et d'informer l'utilisateur en temps réel sur sa position, la connexion et les obstacles. Le chapitre suivant abordera la réalisation logicielle du projet.

Réalisation logicielle

Introduction

Ce chapitre détaille le développement logiciel du projet, depuis la programmation embarquée sur l'ESP32 jusqu'à la création de l'application mobile Flutter. Nous y expliquons les logiques de traitement, de communication et d'affichage mises en place.

1 Programmation embarquée de l'ESP32

La programmation de la carte ESP32 constitue le cœur du système embarqué de la valise autonome. Elle permet à la valise de percevoir son environnement, de prendre des décisions de navigation, et de transmettre les informations nécessaires à l'application mobile Flutter. Les principales fonctionnalités mises en œuvre sont les suivantes :

4.1.1 Lecture des capteurs

Deux types de capteurs sont utilisés :

- Le capteur **ultrasonique** Les données du capteur ultrasonique sont récupérées régulièrement afin de détecter les obstacles et ajuster la trajectoire.
- Le signal **RSSI WiFi** est utilisé pour estimer la distance entre le smartphone de l'utilisateur et la valise, à partir de la puissance du signal WiFi reçu.

4.1.2 Estimation de la distance via le RSSI

Le **RSSI** (Received Signal Strength Indicator) est un indicateur de la puissance du signal WiFi reçu par l'ESP32. Il est exprimé en **dBm**. Une valeur plus proche de zéro indique un signal plus fort. Dans notre projet, il est utilisé pour :

- Estimer la distance entre la valise et le smartphone du propriétaire.
- Déterminer si la valise doit s'approcher, rester sur place, ou signaler une perte de connexion.

Formule de calcul de distance :

$$d = 10^{\frac{A - RSSI}{10n}} \quad (4.1)$$

où :

- d : distance en mètres,
- A : RSSI à 1 mètre,
- n : facteur d'atténuation (selon l'environnement),
- $RSSI$: signal mesuré (en dBm).

4.1.3 Logique de navigation intelligente

La logique de navigation de la valise repose sur l'interprétation croisée des données issues de deux sources : le signal RSSI (Received Signal Strength Indicator) et le capteur ultrasonique. L'objectif est de permettre à la valise de suivre son utilisateur tout en évitant les obstacles, de manière autonome et réactive.

Voici le fonctionnement général de l'algorithme embarqué :

- Lorsque la valise est suffisamment proche de l'utilisateur (selon la valeur du RSSI), elle reste immobile.
- Si le RSSI indique un éloignement (signal plus faible), la valise se met en mouvement pour réduire la distance avec son propriétaire.
- Pendant le déplacement, la valise utilise le capteur ultrasonique pour détecter tout obstacle présent sur son trajet.
- Si un obstacle est détecté, la valise interrompt immédiatement son mouvement et ajuste sa direction pour le contourner.

4.1.4 Logique de détection d'obstacles avec tolérance

Afin d'éviter les fausses alertes lors de la détection d'obstacles, nous avons constaté que le capteur ultrasonique pouvait parfois interpréter de manière incorrecte certains éléments comme étant des obstacles, alors qu'ils ne représentent pas une gêne réelle. Cela peut notamment se produire dans les cas suivants :

- L'utilisateur lui-même (par exemple, une main ou une jambe passant devant le capteur) est détecté comme un obstacle,
- Un objet est détecté à une distance bien supérieure à celle entre la valise et l'utilisateur (selon le RSSI),
- Un obstacle est situé à côté de l'utilisateur mais ne se trouve pas sur la trajectoire de la valise.

Pour résoudre ces problèmes, nous avons mis en place une logique de filtrage basée sur la comparaison entre :

- la distance estimée par le signal RSSI (qui indique à quelle distance se trouve l'utilisateur). et la distance mesurée par le capteur ultrasonique (qui identifie les obstacles physiques).

L'idée est simple : si un obstacle est détecté, mais qu'il est **plus éloigné** que l'utilisateur lui-même, alors il n'est pas considéré comme prioritaire. De plus, une **marge de tolérance** est définie afin d'ignorer les différences minimales de distance dues à des erreurs de mesure ou à des mouvements passagers.

Cette logique est illustrée dans le code embarqué suivant :

- Le signal RSSI est converti en distance estimée à l'aide de la formule standard de propagation de signal.
- La fonction `mesurerDistance()` calcule la distance réelle mesurée par le capteur ultrasonique.
- On considère qu'il y a un véritable obstacle uniquement si :

$$\text{obstacleDistance} < \text{rssiDistance} - \text{TOLERANCE}$$

- Si cette condition n'est pas remplie, le capteur est soit perturbé, soit c'est une fausse alerte, et l'information est ignorée dans la réponse JSON envoyée à l'application Flutter.

Cette logique permet à la valise de naviguer de manière fluide tout en assurant un suivi fiable de l'utilisateur, même dans des environnements dynamiques.

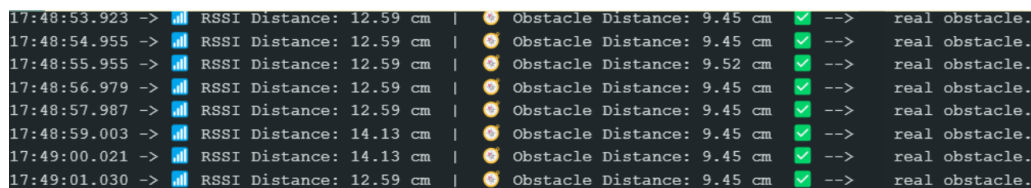
4.1.5 Tests

L'image ci-dessous montre un extrait des résultats issus du moniteur série de l'ESP32 lors d'un test de détection d'obstacles. On y observe les valeurs mesurées en temps réel par le système :

- La distance estimée entre l'utilisateur et la valise, calculée à partir du RSSI, est en moyenne de **12.59 cm**, avec une légère augmentation à **14.13 cm** à deux reprises.
- La distance mesurée par le capteur ultrasonique reste stable autour de **9.45 cm**, ce qui indique la présence constante d'un objet situé devant la valise.

Le système considère l'obstacle comme **réel** et affiche cette détection dans le moniteur série par le message : `-> real obstacle` .

Ce test démontre que la logique de filtrage mise en place fonctionne correctement et permet de détecter efficacement un obstacle légitime tout en évitant les fausses alertes dues à des objets situés en dehors de la trajectoire ou trop proches du propriétaire.



Timestamp	RSSI Distance (cm)	Obstacle Distance (cm)	Detection
17:48:53.923	12.59	9.45	real obstacle.
17:48:54.955	12.59	9.45	real obstacle.
17:48:55.955	12.59	9.52	real obstacle.
17:48:56.979	12.59	9.45	real obstacle.
17:48:57.987	12.59	9.45	real obstacle.
17:48:59.003	14.13	9.45	real obstacle.
17:49:00.021	14.13	9.45	real obstacle.
17:49:01.030	12.59	9.45	real obstacle.

FIGURE 4.1 – Aperçu des résultats de tests

2 Application mobile Flutter

4.2.1 Présentation générale et fonctionnalités

L'application mobile a été développée avec **Flutter**, un framework open source de Google permettant de créer des interfaces modernes, multiplateformes et réactives.

Elle a pour but de permettre à l'utilisateur :

- de savoir si la valise est connectée ou non,
- de connaître la distance entre lui et la valise (calculée à partir du RSSI),
- d'être alerté si un obstacle est détecté,
- de consulter ces informations en temps réel,
- de se déconnecter facilement de l'interface.

Ces fonctions sont accessibles via une interface fluide, organisée autour d'un écran de connexion et d'un tableau de bord principal.

Le diagramme ci-dessous illustre les cas d'utilisation associés à l'application :

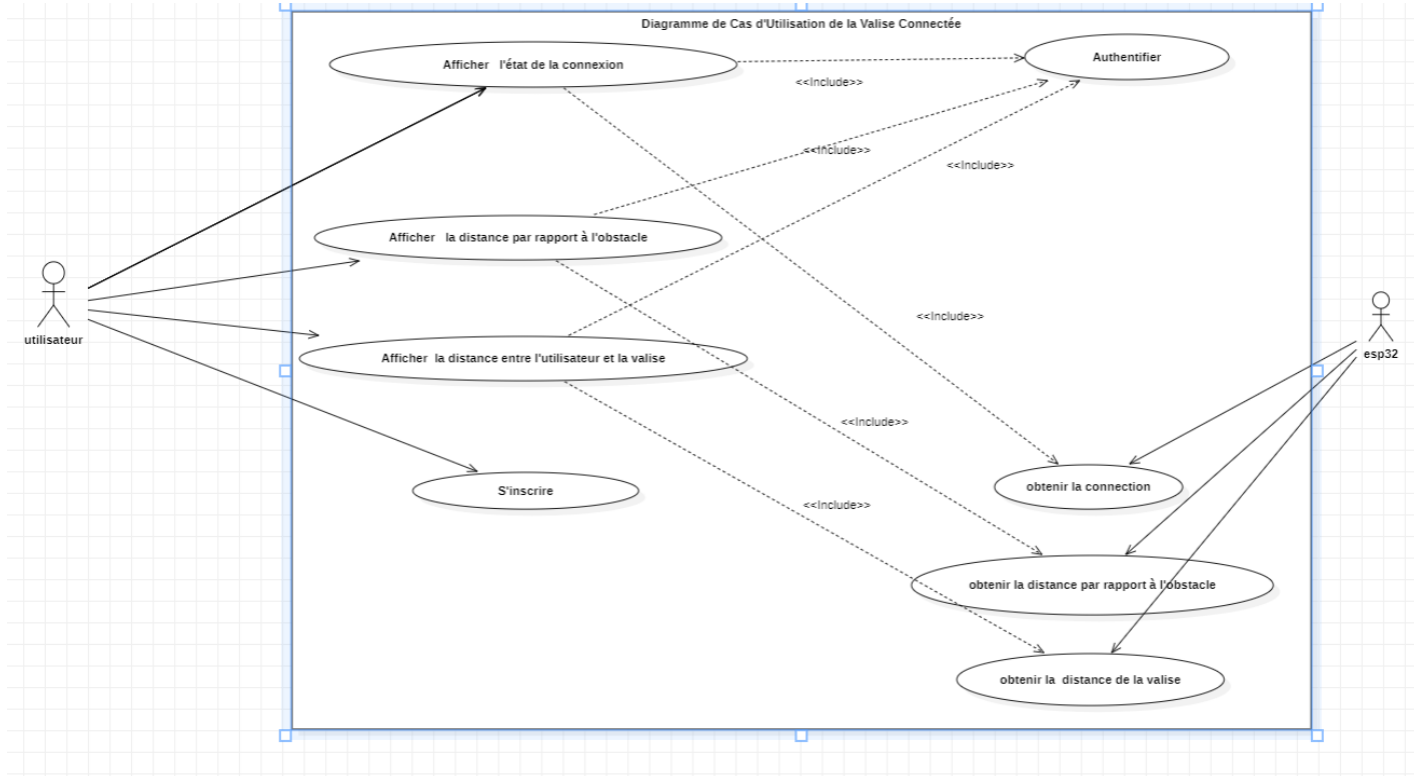


FIGURE 4.2 – Diagramme de cas d'utilisation de l'application mobile Flutter

4.2.2 Architecture et logique de communication

L'application repose sur une architecture inspirée du modèle **MVVM** (Model-View-ViewModel), adaptée à la structure de Flutter :

- **Model** : données reçues depuis l'ESP32 via HTTP (au format JSON),
- **ViewModel** : logique de traitement et d'appel réseau (dans les fichiers `esp32_service.dart`),
- **View** : affichage des données via les interfaces (`home_screen.dart`, `login_screen.dart`).

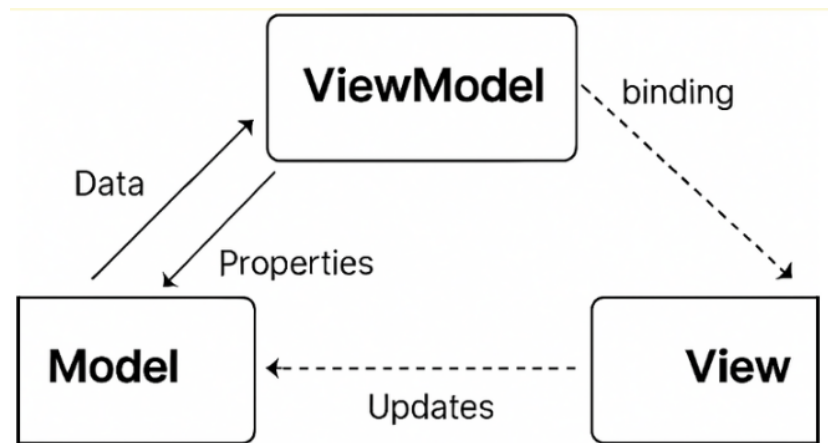


FIGURE 4.3 – Architecture MVVM utilisée dans notre application Flutter

Bien que notre architecture suive le principe général du modèle **MVVM**, nous n'avons pas implémenté de couche *Model* distincte sous forme de classes de données. Les informations reçues depuis l'ESP32, au format **JSON**, sont directement traitées dans les services (notamment dans `esp32_service.dart`), sans être encapsulées dans des objets métiers spécifiques. Ainsi, notre structure se rapproche davantage d'une séparation entre *View* (interfaces utilisateur) et *Service* (logique et appels réseau), sans réelle couche *Model* formelle.

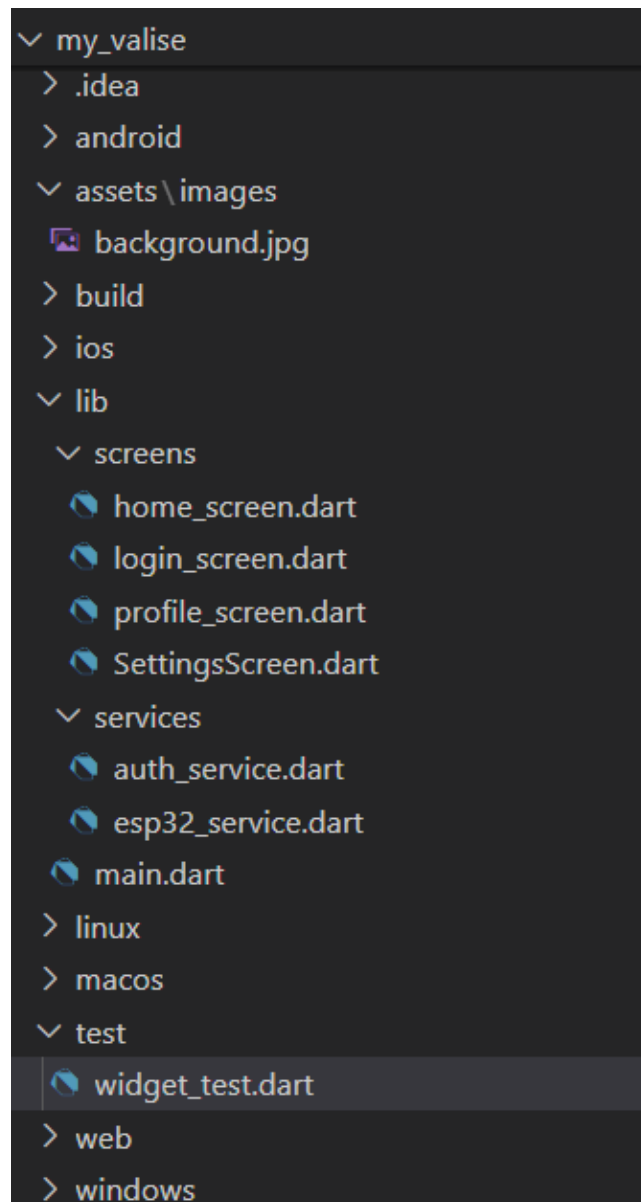


FIGURE 4.4 – Structure des fichiers de l'application Flutter

La communication entre l'application et l'ESP32 se fait via des requêtes HTTP GET envoyées régulièrement vers les endpoints suivants :

- '/status' : statut de connexion.
- '/distance' : distance estimée via RSSI.
- '/obstacle' : distance mesurée par le capteur ultrasonique.

L'ESP32, fonctionnant en mode **Point d'Accès**, répond en JSON avec les valeurs actualisées. Ces données sont ensuite traitées et affichées dynamiquement.

4.2.3 Interface utilisateur

L'interface est constituée de deux écrans principaux :

- **Écran de connexion** L'utilisateur est invité à saisir ses identifiants. Un message d'erreur s'affiche si les champs sont invalides.

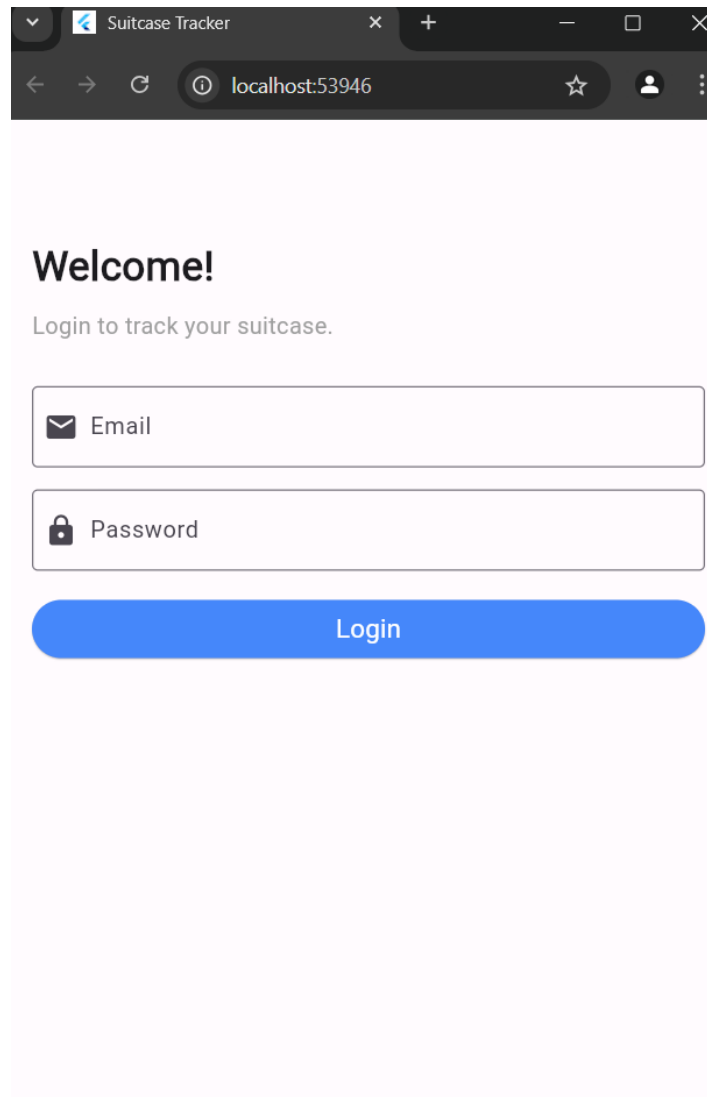


FIGURE 4.5 – Exemple de contrôle de saisie dans l'écran de connexion

- **Écran principal (Home)** Une fois connecté, l'utilisateur accède aux informations en temps réel :
 - distance de la valise,
 - état de la connexion,
 - présence d'obstacles.

Note : L'interface affichée ici correspond à un cas sans connexion active. L'interface en condition réelle a été montrée lors de la démonstration, le matériel n'étant pas disponible au moment de la rédaction.



FIGURE 4.6 – Interface principale de suivi (hors connexion)

— **Déconnexion** L'utilisateur peut quitter l'application à l'aide de la flèche en haut à gauche.

Réalisation matérielle

Introduction

Dans ce chapitre, le travail a principalement porté sur la conception d'un circuit électronique, la gestion de la motorisation via un PCB dédié au contrôle des moteurs, ainsi que l'intégration des capteurs nécessaires au bon fonctionnement du système.

1 Le contrôleur de moteur

5.1.1 Fonctionnement du contrôleur

Le contrôleur moteur utilise un hacheur à quatre quadrants pour gérer la vitesse et le sens de rotation. Les signaux PWM (*Pulse Width Modulation*) sont appliqués aux entrées de contrôle pour ajuster précisément la puissance transmise aux moteurs.

Les images ci-dessous montrent le fonctionnement du contrôleur en fonction des signaux d'entrée *PWM1* et *PWM2*.

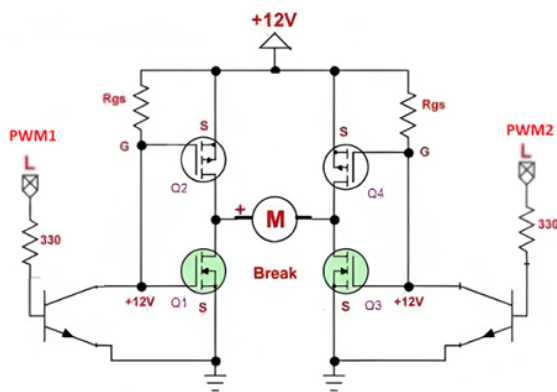


FIGURE 5.1 – Arrêt du moteur lorsque PWM1 et PWM2 sont nuls

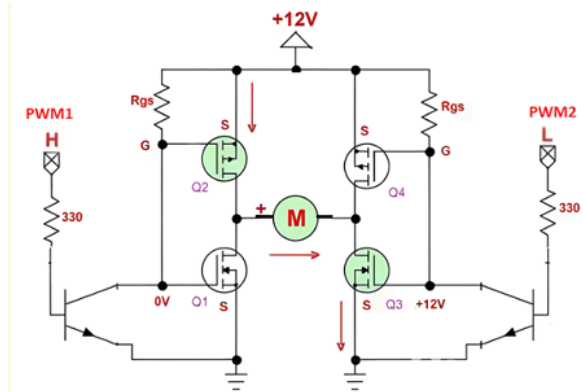


FIGURE 5.2 – Fonctionnement avec PWM1 actif

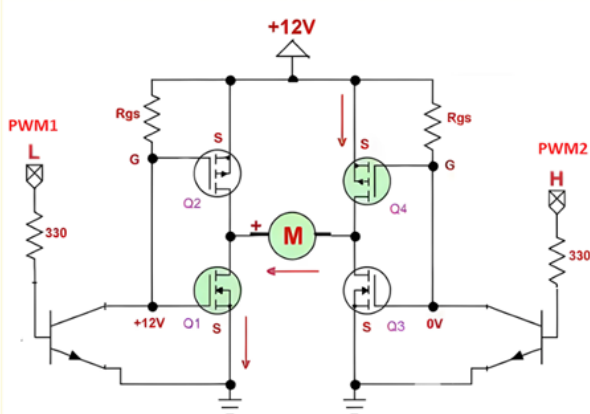


FIGURE 5.3 – Fonctionnement avec PWM2 actif

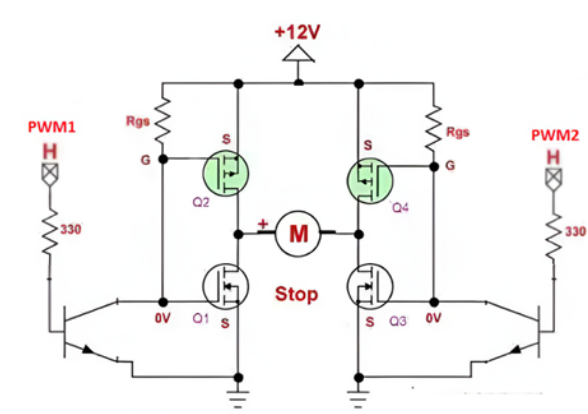


FIGURE 5.4 – Fonctionnement avec PWM1 et PWM2 actifs

Les images ci-dessous montrent le tableau récapitulatif du fonctionnement du notre controlleur :

Input		Output	
PWM1	PWM2	M (+)	M (-)
L	L	L	L
H	L	L	H
L	H	H	L
H	H	OFF (high impedance)	

FIGURE 5.5 – Tableau récapitulatif du fonctionnement du notre controlleur

5.1.2 Dimensionnement du Contrôleur Moteur

Composants Sélectionnés

Les composants principaux sont listés ci-dessous avec leurs caractéristiques respectives, suivant le modèle de tableau fourni :

TABLE 5.1 – Caractéristiques des composants du contrôleur moteur

Composant	Caractéristiques
Motoréducteur FIT0564	Tension nominale : 12 V Courant nominal : 1 A Courant maximal : 5 A
MOSFET P-Channel IRF4905 (Q1 et Q2)	$R_{DS(on)} = 0.02 \Omega$ $I_{max} = 74 A$ Compatible pour une alimentation de 12 V
MOSFET N-Channel IRFP044N (Q3 et Q4)	$R_{DS(on)} = 0.022 \Omega$ $I_{max} = 57 A$
Transistors bipolaires (Q5 et Q6) 2N2222	$V_{CE(max)} = 40 V$ $I_{C(max)} = 800 mA$ $V_{EB(max)} = 6 V$ $\beta = 100$

Simulation du Contrôleur

La conception a été modélisée et simulée avec **Proteus**. Les résultats ont validé les points suivants :

- Régulation précise de la vitesse via PWM.
- Contrôle bidirectionnel des moteurs.
- Fonctionnement stable sous différentes charges.

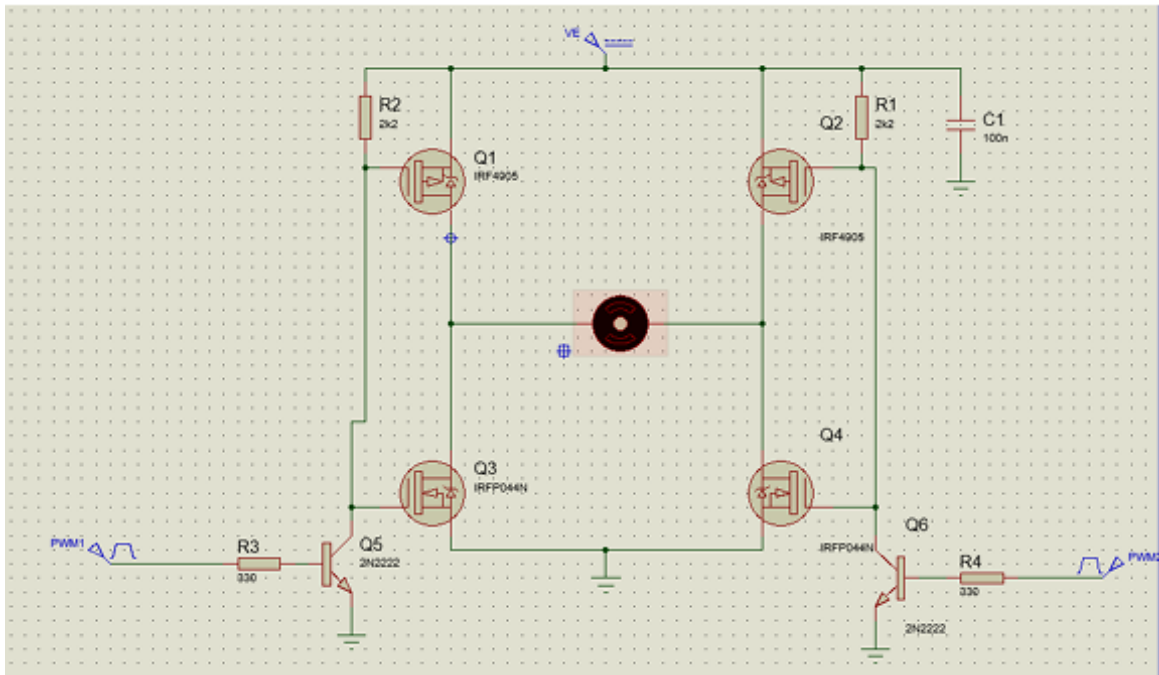


FIGURE 5.6 – Simulation : Contrôle bidirectionnel du moteur

Essai du circuit sur plaque d'essai

Avant de lancer le routage du PCB, une première version du circuit a été montée sur une plaque d'essai (breadboard). Cette étape a permis de tester le fonctionnement global du système de commande moteur avec les composants principaux : pont en H, ESP32, alimentation et moteurs. Les essais réalisés ont permis de :

- Vérifier que le contrôle des moteurs via des signaux PWM fonctionnait correctement.
- Tester l'inversion du sens de rotation des moteurs en envoyant des commandes à l'ESP32.
- Tester les alimentations séparées pour l'ESP32 (3.3V) et les moteurs (12V).

Les résultats ont été concluants, car le système répondait aux commandes comme prévu. Cette phase de validation sur plaque d'essai a permis de confirmer le bon fonctionnement du circuit avant de procéder à la conception du PCB.

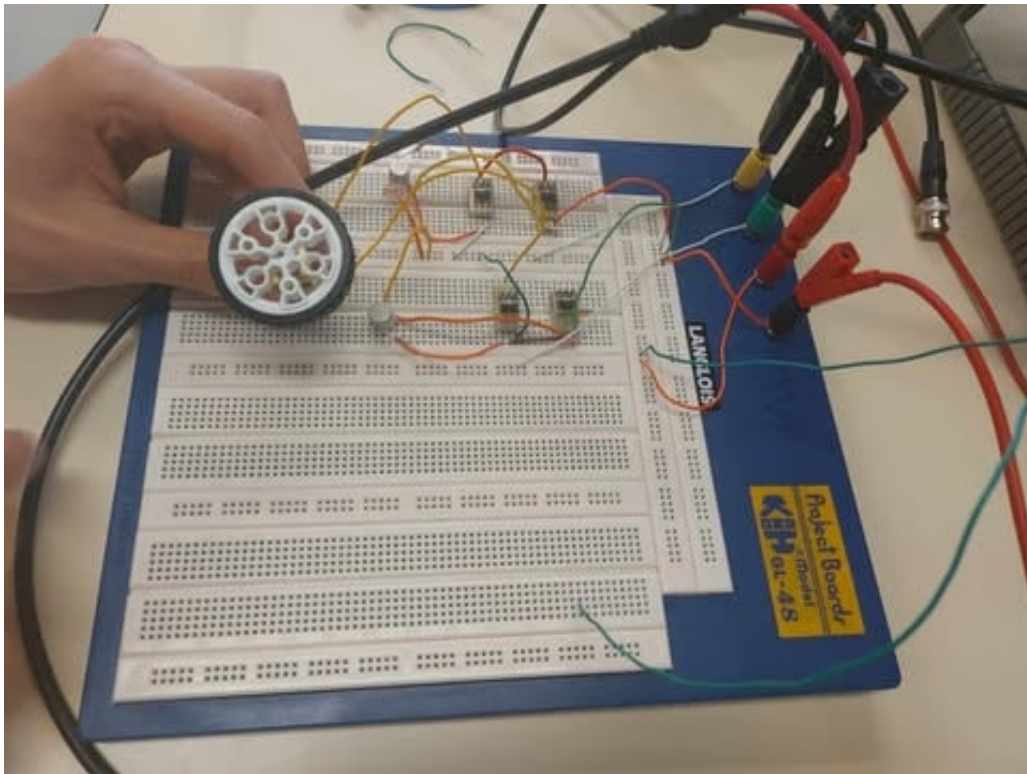


FIGURE 5.7 – Essai du circuit de contrôle moteur

5.1.3 Routage du PCB sur KiCad

Après validation du schéma électrique, le routage du circuit imprimé a été réalisé à l'aide du logiciel KiCad. L'objectif était de concevoir un PCB compact, fiable et adapté aux contraintes de puissance liées au pilotage des moteurs.

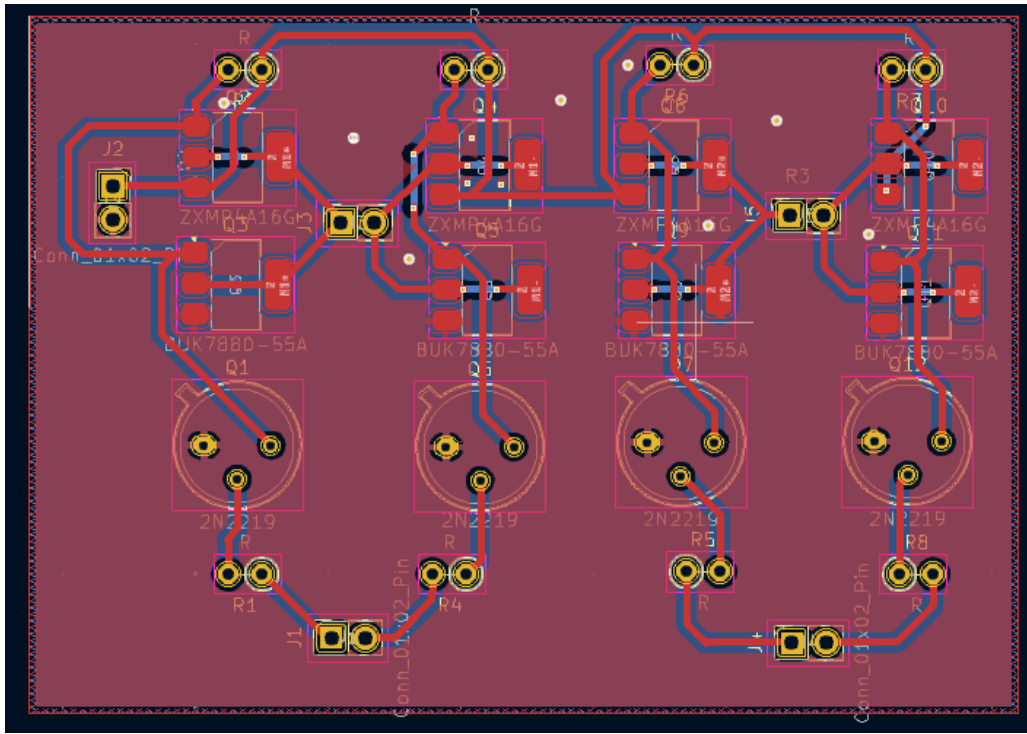


FIGURE 5.8 – Routage du PCB sur KiCad

Le placement des composants a été optimisé pour :

- Réduire la longueur des pistes de puissance.
- Limiter les interférences entre les signaux de commande et les circuits de puissance.
- Faciliter les connexions avec les moteurs, l'alimentation et la carte de contrôle.

Des plans de masse ont été ajoutés pour améliorer la stabilité électrique du circuit. Les pistes d'alimentation ont été élargies pour supporter le courant nécessaire aux moteurs (figure ci-dessous). Des vias thermiques et des zones de dissipation ont également été prévues autour du pont en H pour éviter la surchauffe.

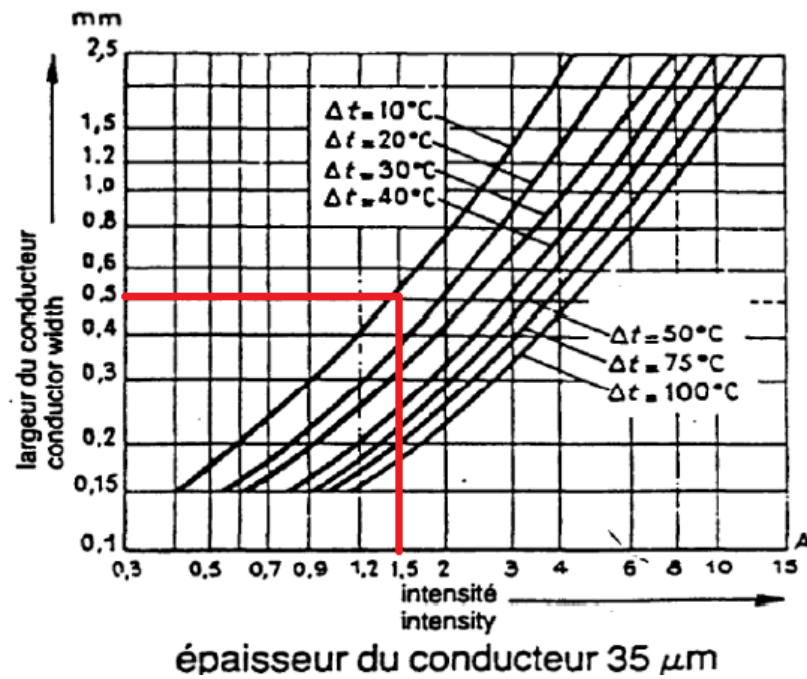


FIGURE 5.9 – largeur des pistes

5.1.4 Impression du PCB et tests

Une fois le routage terminé, les fichiers Gerber ont été générés pour la fabrication du circuit. Le PCB a été fabriqué à partir des fichiers Gerber, puis les composants ont été soudés manuellement. Après assemblage, plusieurs tests ont été réalisés :

- Test de continuité pour vérifier l'intégrité des pistes.
- Test de tension sans charge pour contrôler la bonne distribution de l'alimentation.
- Commande moteur via l'ESP32 pour tester les signaux de commande et la réponse des moteurs.

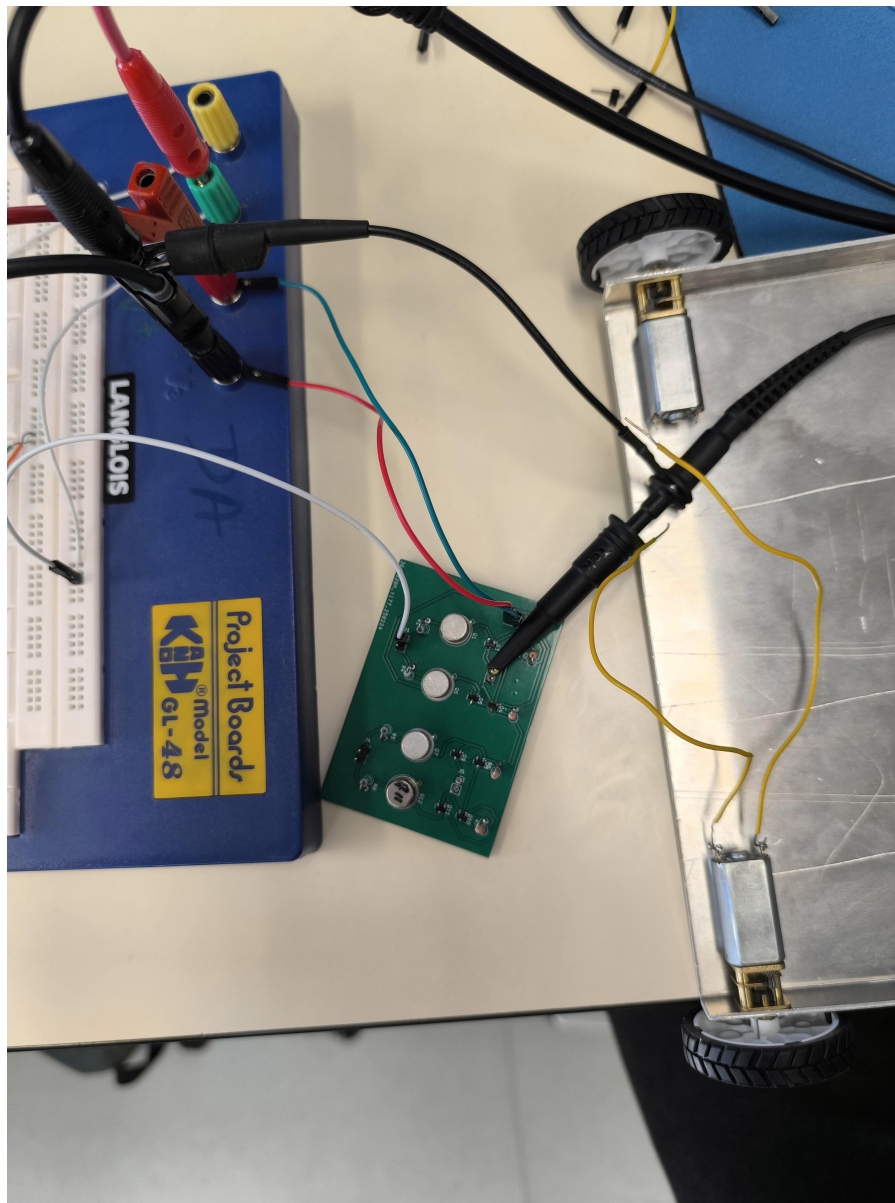


FIGURE 5.10 – les tests

Lors des premiers essais, nous avons rencontré un dysfonctionnement au niveau du circuit : les moteurs ne répondaient pas correctement aux signaux de commande, et certains composants ne réagissaient pas comme prévu. Après plusieurs vérifications (soudure, connexions, tensions), il s'est avéré que le problème provenait d'une erreur dans le choix des empreintes .

Une version corrigée du schéma a alors été réalisée. Toutefois, faute de temps, nous n'avons pas pu imprimer la nouvelle version du PCB avant la fin du projet. Le montage final repose donc sur la version initiale avec des ajustements manuels, bien que la version corrigée soit prête à être produite.

2 Intégration des Capteurs et alimentation

Intégration des Capteurs et Alimentation L'intégration des capteurs et du système d'alimentation est une étape clé pour assurer la fonctionnalité et l'autonomie de la valise. En combinant les capteurs nécessaires au suivi et à la détection des obstacles avec un système d'alimentation efficace, la valise peut fonctionner de manière autonome et réactive.

5.2.1 Capteur ultrasonique SH04

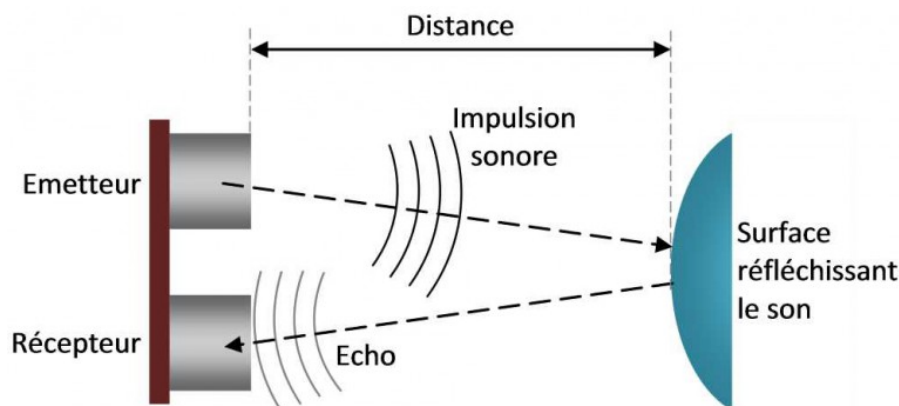
Le capteur ultrasonique SH04 a été choisi pour sa capacité à mesurer la distance entre la valise et les objets qui se trouvent devant elle, permettant ainsi d'éviter les obstacles.

Calcul de la distance

Le calcul de la distance se base sur le principe de la vitesse du son dans l'air. Le temps mesuré entre l'émission de l'onde et sa réception est multiplié par la vitesse du son, puis divisé par deux pour obtenir la distance entre le capteur et l'obstacle. La formule est la suivante :

$$\text{Distance} = \frac{\text{Vitesse du son} \times \text{Temps}}{2}$$

- **Vitesse du son (v)** : La vitesse du son dans l'air est généralement considérée comme 343 mètres par seconde (m/s) à température ambiante (20°C).
- **Temps (t)** : Le temps mesuré par le capteur entre l'émission de l'onde et la réception de son écho. Ce temps est généralement très court, mesuré en microsecondes (µs).



Cette distance calculée est ensuite utilisée pour déterminer si la valise doit ajuster sa trajectoire afin d'éviter un obstacle.

Les avantages de ce capteur sont :

- **Précision** : il offre une bonne précision pour les distances courtes à moyennes.
- **Faible consommation d'énergie** : il est adapté pour des systèmes alimentés par batterie.
- **Simplicité d'intégration** : son interface de communication est simple à utiliser avec l'ESP32, ce qui permet une intégration rapide.

Le capteur SH04 a été positionné à l'avant de la valise pour surveiller son environnement immédiat et éviter les collisions avec des objets.

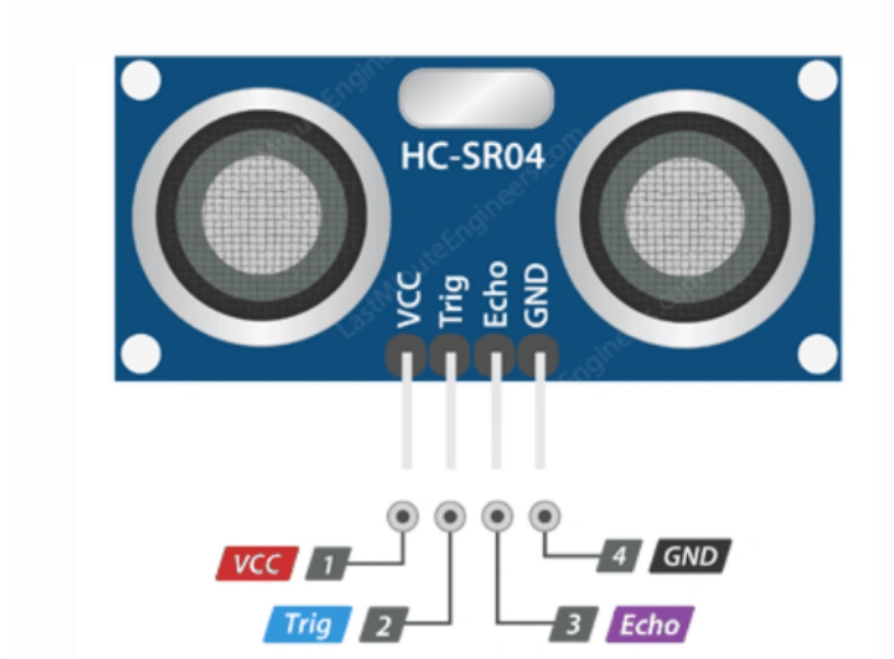


FIGURE 5.11 – Capteur ultrasonique SH04

3 Châssis et structure mécanique

Le châssis constitue la base physique de la valise autonome, assurant la fixation stable et sécurisée de l'ensemble des composants matériels : moteurs, capteurs, batteries, carte ESP32 et contrôleur. Dans ce projet, nous avons conçu un châssis personnalisé en métal, adapté à notre besoin en termes de robustesse, de stabilité et d'intégration mécanique.

5.3.1 Conception et matériau du châssis

Le châssis a été réalisé à partir de plaques métalliques, découpées et assemblées manuellement. Ce choix de matériau garantit une structure rigide, capable de supporter les vibrations liées au déplacement et les contraintes mécaniques.

La conception a été guidée par plusieurs critères :

- **Résistance mécanique** suffisante pour supporter l'ensemble du système (batterie, moteurs, capteurs, électronique),
- **Stabilité dynamique** pendant les phases de déplacement,
- **Facilité de fixation** pour les composants avec perçage ou vissage direct,
- **Format compact et optimisé** pour respecter l'encombrement d'une valise réelle.

5.3.2 Dimensionnement et implantation des composants

Les dimensions globales du châssis ont été fixées à :

- **300 mm (longueur) × 200 mm (largeur).**

La disposition des composants a été pensée pour équilibrer les masses et optimiser les performances :

- **Batterie** : placée au centre pour une meilleure répartition du poids.
- **ESP32 et contrôleur moteur** : positionnés à l'arrière, à proximité des moteurs, afin de réduire la longueur des câbles de puissance.
- **Capteurs ultrasoniques** : fixés à l'avant pour maximiser la couverture de détection d'obstacles.
- **Moteurs DC** : montés à l'arrière sur des supports métalliques prévus à cet effet.
- **Roues avant** : fixes ou pivotantes selon les essais réalisés, placées à l'avant du châssis.

Conclusion

La conception du contrôleur de moteur, associée à un dimensionnement précis des composants, a permis de garantir une commande efficace et sécurisée des moteurs. La simulation a validé le bon fonctionnement du circuit avant la réalisation du PCB. L'intégration des capteurs et de l'alimentation assure une autonomie et une réactivité adaptées aux besoins de la valise. Cette étape constitue une base technique solide pour le déplacement autonome du système.

Conclusion générale

Ce projet de valise autonome connectée s'inscrit dans une démarche d'innovation visant à améliorer la gestion des bagages et la sécurité lors des déplacements. Il repose sur l'interaction entre un système embarqué (ESP32), des capteurs physiques, une logique de navigation intelligente, et une application mobile Flutter.

Tout au long du développement, nous avons été confrontés à divers problèmes techniques : instabilité du protocole TCP, variations imprévisibles du signal RSSI, imprécisions du capteur ultrasonique, incompatibilité des bibliothèques, ou encore difficultés dans la fabrication du circuit imprimé du contrôleur moteur. Pour chaque obstacle rencontré, nous avons trouvé une solution adaptée, comme l'intégration d'une tolérance entre la mesure RSSI et la distance capteur, ou le passage à une communication HTTP plus stable.

Malgré les nombreuses difficultés rencontrées, nous avons tout de même réussi à faire fonctionner plusieurs parties essentielles du projet. Bien que le prototype complet ne soit pas finalisé, nous avons pu tester et valider des fonctionnalités clés telles que la communication entre l'ESP32 et l'application Flutter, l'estimation de la distance via le RSSI, ainsi que la détection d'obstacles.

Ce projet nous a permis d'acquérir des compétences solides et variées :

— **Côté logiciel :**

- Développement d'une application Flutter communiquant avec un microcontrôleur,
- Intégration d'une logique de traitement des données, de gestion d'état et d'affichage en temps réel,
- Implémentation d'un serveur HTTP sur ESP32 avec traitement des requêtes JSON,
- Test et adaptation du RSSI pour estimer la distance.

— **Côté matériel :**

- Programmation embarquée (capteurs, moteur, logique de navigation),
- Simulation, prototypage et test du contrôleur moteur,
- Conception et routage du PCB via KiCad,
- Intégration d'une logique de détection d'obstacle avec marge de tolérance.
- **Compétences transversales :**
 - Gestion de projet (diagrammes de Gantt, répartition des tâches),
 - Capacité à faire face à des imprévus techniques,
 - Documentation, collaboration et présentation du projet en démonstration réelle.

Sur le plan personnel, ce projet nous a appris à collaborer efficacement, à documenter rigoureusement chaque étape, et à ne pas sous-estimer l'importance des tests en conditions réelles. Si c'était à refaire, nous structurerions l'application Flutter avec un modèle MVVM complet dès le début, et nous anticiperions davantage les contraintes liées à la conception du PCB pour éviter les erreurs de composants ou de routage.

En conclusion, ce projet a été une aventure technique, humaine et pédagogique. Il nous a permis de mettre en pratique nos acquis tout en développant des compétences nouvelles dans un contexte concret, riche en enseignements. Il représente pour nous une étape importante dans notre préparation au monde professionnel.

Bibliographie

- [1] ForwardX Ovis – *Valise autonome basée sur l'IA et la caméra.*
Disponible sur : <https://www.forwardx.com/ovis/> [Consulté le 16 Décembre 2024].
- [2] Travelmate Robotics – *Valise autonome avec Bluetooth RSSI.*
Disponible sur : <https://travelmate.robotics/> [Consulté le 16 Décembre 2024].
- [3] Cowarobot R1 – *Valise autonome utilisant des capteurs ultrasoniques et LiDAR.*
Disponible sur : <https://www.cowarobot.com/> [Consulté le 16 Décembre 2024].
- [4] Capteur ultrasonique HC-SR04 – *Détection d'obstacles.*
Disponible sur : <https://www.gotronic.fr/art-module-de-detection-us-hc-sr04-20912.htm>
[Consulté le 16 Décembre 2024].
- [5] NodeMCU ESP32 – *Microcontrôleur compact avec Wi-Fi et Bluetooth.*
Disponible sur : <https://www.gotronic.fr/art-module-nodemcu-esp32-28407.htm> [Consulté le 16 Décembre 2024].
- [6] Paire de roues FIT0494 – *Accessoires pour robotique.*
Disponible sur : <https://www.gotronic.fr/art-paire-de-roues-fit0494-37019.htm> [Consulté le 16 Décembre 2024].

Résumé

Dans ce rapport, nous avons présenté notre avancement dans le projet qui concerne la conception et la réalisation d'une valise autonome capable de suivre son propriétaire . Ce travail inclut une partie de gestion de projet, l'étude de l'environnement, l'architecture du projet, et finalement la réalisation.

Mots clés : Valise autonome, Navigation intelligente, Gestion de projet, Étude de l'environnement, Architecture, Réalisation.

Abstract

In this report, we present our progress on the project concerning the design and implementation of an autonomous suitcase capable of following its owner . This work includes a project management section, an environment study, the system architecture, and finally the implementation.

Keywords : Autonomous suitcase, Intelligent navigation, Project management, Environment study, Architecture, Implementation.